Introduction to Angular

Lecture 4

Angular v21 • TypeScript • Angular CLI

KBTU

1. Introduction to Angular

What is Angular?

- Open-source front-end framework developed and maintained by Google
- Used for building Single Page Applications (SPAs)
- Written in TypeScript
- Current stable version: Angular 21 (2025)
- Component-based architecture – reusable, self-contained UI building blocks

History & Evolution

Year	Version	Key Changes
2010	AngularJS	Original version, JavaScript-based, MVC pattern
2016	Angular 2	Complete rewrite in TypeScript, component-based
2023	Angular 17	Standalone components default, new control flow (@if, @for)
2024	Angular 18–19	Signals stable, zoneless change detection (experimental)
2025	Angular 20–21	Signal forms, Vitest default, old *ngIf/*ngFor removed

Why Choose Angular?

- **Full-featured framework** – routing, forms, HTTP client, testing – all built-in
- **Strong typing** – TypeScript catches errors at compile time
- **Component-based** – modular, reusable UI building blocks
- **Large ecosystem** – Angular Material, CDK, extensive third-party libraries
- **Enterprise adoption** – long-term support from Google

---

2. Goals and Architecture

Core Goals

1. Component-based architecture – modular, reusable UI pieces
2. Separation of concerns – templates (HTML), logic (TS), and styles (CSS) separate
3. Testability – designed with unit testing in mind
4. Cross-platform – web, mobile (Ionic), desktop (Electron)
5. Performance – ahead-of-time compilation, tree-shaking, Signals reactivity

Key Building Blocks

Concept	Description	Example
Components	UI blocks with template + logic + styles	AppComponent, HeaderComponent
Templates	HTML with Angular syntax	{{ title }}, (click), @if, @for
Services	Reusable business logic (future lecture)	UserService, AuthService
Dependency Injection	Provides dependencies automatically	constructor(private svc: MyService)
Pipes	Transform data in templates	{{ date | date:'short' }}
Router	Handles navigation between views	RouterModule, app.routes.ts

How it all connects:

User interacts with Template → Template is bound to Component → Component uses Services for data/logic → Router handles navigation between Components.

---

3. Angular CLI Usage

The Angular CLI (@angular/cli) is a command-line tool that automates project creation, development, testing, and builds.

Installation

```
# Install Angular CLI globally
npm install -g @angular/cli

# Verify installation
ng version
```

Creating Your First Project

```
# Create new project
ng new my-first-app

# CLI will ask:
#   Which stylesheet format? (CSS / SCSS / Sass / Less)
#   Enable SSR? (y/N)

# Navigate and start dev server
cd my-first-app
ng serve --open
```

The dev server starts at http://localhost:4200 with hot module replacement (HMR) – style and template changes appear instantly.

Essential Commands

Command	Description
`ng new project-name`	Create a new Angular project
`ng serve`	Start development server (localhost:4200)
`ng serve --open`	Start server and open browser
`ng generate component name`	Generate a new component
`ng generate service name`	Generate a new service
`ng generate pipe name`	Generate a new pipe
`ng build`	Build for production
`ng test`	Run unit tests

Shorthand Commands

```
ng g c component-name       # Generate component
ng g s service-name         # Generate service
ng g p pipe-name            # Generate pipe
ng g d directive-name       # Generate directive
ng g i models/user          # Generate interface
```

CLI Cheat Sheet

```
# Project setup with options
ng new my-app --style=scss        # With SCSS

# Generate into subfolders
ng g c components/header           # Component in folder
ng g s services/auth               # Service in folder

# Development
ng serve --port 4201               # Custom port

# Build & other
ng build                           # Production build
ng update                          # Update dependencies
ng add @angular/material           # Add librar
```

4. JavaScript vs TypeScript Basics

Angular uses TypeScript, a superset of JavaScript that adds static typing. Any valid JavaScript is valid TypeScript, but TypeScript adds features for safety, tooling, and maintainability.

Why TypeScript for Angular?

Feature	JavaScript	TypeScript
Type system	Dynamic (runtime)	Static (compile-time)
Type annotations	No	Yes: string, number, boolean, etc.
Interfaces	No	Yes
Access modifiers	No (ES2022 has #private)	public, private, protected
Enums	No	Yes
Decorators	Proposal stage	Full support (used in Angular)
Compile-time errors	No	Yes
IDE support	Basic	Rich autocompletion, refactoring

Type Annotations

```
// Basic types
let name: string = 'Angular';
let version: number = 21;
let isActive: boolean = true;
let items: string[] = ['a', 'b', 'c'];

// Any type (avoid when possible)
let data: any = 'could be anything';

// Union types
let id: string | number = 123;

// Type inference (TypeScript figures it out)
let inferred = 'TypeScript knows this is a string';
```

Functions

JavaScript:

```
function greet(name) {
  return "Hello, " + name;
}
```

TypeScript – typed parameters and return type:

```
function greet(name: string): string {
  return `Hello, ${name}`;
}
```

Interfaces

Interfaces define the shape of objects. They exist only at compile time and help catch errors early.

```
interface User {
  id: number;
  name: string;
  email: string;
  age?: number;            // optional property
  readonly createdAt: Date; // cannot be modified
}

const user: User = {
  id: 1,
  name: 'John Doe',
  email: 'john@example.com',
  createdAt: new Date()
};
```

Classes

```
class UserService {
  private users: User[] = [];
  public apiUrl: string = '/api/users';

  constructor(private http: HttpClient) {}

  getUsers(): User[] {
    return this.users;
  }

  addUser(user: User): void {
    this.users.push(user);
  }

  findById(id: number): User | undefined {
    return this.users.find(u => u.id === id);
  }
}
```

Enums and Type Aliases

```
// Enum definition
enum CourseLevel {
  Beginner = 'BEGINNER',
  Intermediate = 'INTERMEDIATE',
  Advanced = 'ADVANCED'
}

// Type alias (union of literal types)
type StudentStatus = 'enrolled' | 'graduated' | 'suspended';
```

Decorators (Essential for Angular)

Decorators add metadata to classes. Angular uses them to define components, services, and more.

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {
  title = 'My Angular App';
}

// Service decorator
@Injectable({
  providedIn: 'root'
})
export class DataService {
  // service logic
}
```

---

5. Angular Template Syntax

In Angular 17+, the new built-in control flow syntax (@if, @for, @switch) replaced the old structural directives (*ngIf, *ngFor, *ngSwitch). No CommonModule import needed.

Old vs New Syntax

Feature	Old Syntax (pre-v17)	New Syntax (v17+)
Conditional	`*ngIf="isOk"`	`@if (isOk) { ... }`
Loop	`*ngFor="let x of items"`	`@for (x of items; track ...) { }`
Switch	`[ngSwitch] / *ngSwitchCase`	`@switch (val) { @case (...) }`
Import	Requires CommonModule	No import needed

Template Syntax Overview

- **Interpolation {{ }}** – displays a component property value in the template
- **Property Binding [ ]** – binds a component property to an element property: [src]="imageUrl"
- **Event Binding ( )** – connects a DOM event to a component method: (click)="doSomething()"
- **Two-way Binding [( )]** – combines property + event binding: [(ngModel)]="name" (requires FormsModule)
- **@if / @else** – conditionally renders elements based on a boolean expression

- **@for ... track** – loops over an array; track expression is required for performance
- **@switch / @case** – renders one of several options based on a value

@for – The track Expression

Unlike the old *ngFor, the new @for requires a track expression. This tells Angular how to identify each item for efficient DOM updates.

```
// Track by index (simple cases)
@for (num of numbers; track $index) {
  <li>{{ num }}</li>
}

// Track by unique property (recommended for objects)
@for (student of students; track student.id) {
  <li>{{ student.name }}</li>
}

// Available implicit variables:
// $index, $first, $last, $even, $odd, $count
```

@if / @else Example

```
@if (isOk) {
  <p>Everything is fine!</p>
} @else {
  <p>Something went wrong.</p>
}

// Multiple conditions:
@if (score >= 90) {
  <p>Grade: A</p>
} @else if (score >= 70) {
  <p>Grade: B</p>
} @else {
  <p>Grade: C</p>
}
```

@for with @empty

```
@for (item of items; track item.id) {
  <div>{{ item.name }}</div>
} @empty {
  <p>No items found.</p>
}
```

@switch Example

```
@switch (selectedStatus) {
  @case ('pending') {
    <p>Status: Pending</p>
  }
```

```
@case ('active') {
  <p>Status: Active</p>
}
@case ('completed') {
  <p>Status: Completed</p>
}
@default {
  <p>Unknown status</p>
```

6. Practical Demos

Demo 1: Hello World Component

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <div>
      <h1>{{ title }}</h1>
      <p>Welcome to {{ framework }}!</p>
    </div>
  `,
  styles: [`h1 { color: #dd0031; }`]
})
export class AppComponent {
  title = 'Hello, Angular!';
  framework = 'Angular 21';
}
```

Demo 2: Data Binding Examples

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-data-binding',
  imports: [FormsModule],
  template: `
    <div>
      <!-- 1. Interpolation (Component -> Template) -->
      <h3>Interpolation {{ }}</h3>
      <p>Hello, {{ userName }}!</p>
      <p>2 + 2 = {{ 2 + 2 }}</p>
      <p>Uppercase: {{ userName.toUpperCase() }}</p>

      <!-- 2. Property Binding (Component -> Template) -->
      <h3>Property Binding [ ]</h3>
      <img [src]="imageUrl" [alt]="imageAlt" width="100">
      <button [disabled]="isDisabled">Click Me</button>

      <!-- 3. Event Binding (Template -> Component) -->
      <h3>Event Binding ( )</h3>
      <button (click)="onClick()">Click Counter: {{ clickCount }}</button>

      <!-- 4. Two-way Binding -->
      <h3>Two-way Binding [( )]</h3>
      <input [(ngModel)]="twoWayText" placeholder="Two-way binding">
      <p>Value: {{ twoWayText }}</p>
    </div>
  `
})
export class DataBindingComponent {
```

```
  userName = 'Student';
  imageUrl = 'https://angular.dev/assets/images/press-kit/angular_icon.svg';
  imageAlt = 'Angular Logo';
  isDisabled = false;
  clickCount = 0;
  twoWayText = '';

  onClick(): void {
    this.clickCount++;
  }
}
```

Demo 3: Control Flow (@if, @for, @switch)

```
import { Component } from '@angular/core';

interface Product {
  id: number;
  name: string;
  price: number;
  inStock: boolean;
}

@Component({
  selector: 'app-directives',
  template: `
    <div>
      <h2>Control Flow Demo</h2>

      <button (click)="toggleVisibility()">Toggle Content</button>

      @if (isVisible) {
        <p>Content is visible!</p>
      } @else {
        <p>Content is hidden</p>
      }

      <h4>Product List:</h4>
      <ul>
        @for (product of products; track product.id) {
          <li>
            {{ product.name }} - \${{ product.price }}
            @if (product.inStock) {
              <span>In Stock</span>
            } @else {
              <span>Out of Stock</span>
            }
          </li>
        } @empty {
          <li>No products available</li>
        }
      </ul>

      <!-- @switch example -->
      <select (change)="onStatusChange($event)">
        <option value="pending">Pending</option>
        <option value="active">Active</option>
```

```
        <option value="completed">Completed</option>
      </select>

      @switch (selectedStatus) {
        @case ('pending') { <p>Pending</p> }
        @case ('active') { <p>Active</p> }
        @case ('completed') { <p>Completed</p> }
      }
    </div>
  `
})
export class DirectivesComponent {
  isVisible = true;
  selectedStatus = 'pending';

  products: Product[] = [
    { id: 1, name: 'Laptop', price: 999, inStock: true },
    { id: 2, name: 'Phone', price: 699, inStock: true },
    { id: 3, name: 'Tablet', price: 499, inStock: false },
    { id: 4, name: 'Watch', price: 299, inStock: true },
  ];

  toggleVisibility(): void {
    this.isVisible = !this.isVisible;
  }

  onStatusChange(event: Event): void {
    this.selectedStatus = (event.target as HTMLSelectElement).value;
  }
}
```

7. Practice Exercises

Exercise 1: Create a new Angular project using the CLI. Run it locally and verify you see the default page at localhost:4200.

Exercise 2: Modify AppComponent to display your name and a list of your favorite programming languages using @for.

Exercise 3: Add a button that toggles a greeting message on and off using @if and a boolean property.

Exercise 4: Create a counter component: display a number, add "+" and "–" buttons to increment and decrement it, and use @if to show a warning when the counter goes below 0.

Exercise 5: Create an array of Student objects (with id, name, grade). Display them using @for with track student.id. Use @if to highlight students with grade >= 90.

Exercise 6: Create a data binding demo: use interpolation to display a property, property binding to set an image src, event binding for a click counter, and two-way binding with an input field.

Summary

Topic	Key Takeaways
Angular	Full-featured SPA framework by Google, current version 21
Architecture	Component-based with DI, services, pipes, routing
CLI	Powerful tool for scaffolding, development, and building
TypeScript	Static typing, interfaces, classes, enums, decorators
Templates	New control flow: @if, @for, @switch (replaces *ngIf, *ngFor)
Data Binding	Interpolation {{ }}, property [ ], event ( ), two-way [( )]